Rapport : Système de réservation de salles

1. Introduction

Ce projet consiste en la création d'un système de gestion de réservations pour les ressources d'une université (salles de cours). L'application permet de gérer des élèves, de vérifier la disponibilité des salles en temps réel et d'appliquer des règles métier spécifiques pour assurer le bon fonctionnement du service.

2. Structure du Code (POO)

Conformément à l'étape 3 du projet, le code a été structuré en Programmation Orientée Objet pour une meilleure modularité.

Description des Classes

L'application s'articule autour de trois classes principales :

Classe	Rôle	Attributs Clés
Salle	Représente la ressource physique.	numéro
Eleve	Représente l'utilisateur du système.	nom, prénom, penalites
Reservation	Lie un élève à une salle sur un créneau précis.	salle, jour, heure_debut, heure_fin, eleve_id

Méthodes Clés

- **Eleve.peut_reserver()** : Vérifie si l'élève a le droit de créer une réservation en fonction de ses points de pénalité.
- **Reservation.chevauche()** : Compare deux créneaux horaires pour détecter un conflit de réservation.
- **Eleve.nom_complet()** : Retourne l'identité formatée de l'élève pour l'affichage.

3. Règles Métier Implémentées

Pour garantir la cohérence des données, plusieurs règles :

1. Disponibilité Temporelle : Les réservations ne sont possibles qu'entre 8h et 18h.
2. Anti-Collision : Le système empêche deux réservations sur la même salle, le même jour, si les horaires se chevauchent.
3. Système de Sanction : Un élève ayant cumulé 50 points de pénalité ou plus est automatiquement bloqué par le système.
4. Validation des Salles : Seules les salles définies dans la configuration (101-105, 201-205) peuvent être réservées.

4. Persistance des Données

Pour répondre à l'étape 5, l'état du programme est sauvegardé dans un fichier db.json.

- Au démarrage : Le programme charge les objets (élèves, réservations, historique) depuis le fichier JSON.
- À chaque action : Chaque modification (nouvelle réservation, ajout de pénalité) déclenche une mise à jour du fichier pour éviter toute perte de données.

5. Fonctionnalités Avancées (Bonus)

Nous avons enrichi le système avec plusieurs modules supplémentaires:

- Interface Graphique : Développée avec Tkinter pour faciliter l'utilisation par les administrateurs.
- Statistiques : Analyse automatique de la salle la plus fréquentée et du jour le plus chargé.
- Historique : Journal complet de toutes les actions effectuées (créations, annulations, pénalités).
- Tests Automatisés : Un module de tests intégré permet de vérifier la fiabilité des algorithmes de calcul.

FONCTION PRINCIPALES

- nom_complet() (main.py (line 70))
Utilisée partout pour afficher proprement le prénom + nom des élèves (tableau, historique, pénalités, listes déroulantes).

- rafraichir_tableau() (main.py (line 39))
Met à jour la table des réservations après chargement, création ou annulation.

- salle_est_disponible() (main.py (line 180))
Vérifie qu'une salle est libre sur un créneau (appel métier + appels dans les tests).

- sauvegarder_donnees() (main.py (line 200))
Sauvegarde l'état complet (réservations, élèves, historique, compteur) après chaque modification.

- verifier_droit_reservation() (main.py (line 175))
Contrôle si un élève peut réserver (bloqué si trop de pénalités).

- les_horaires_se_chevauchent() (main.py (line 556))
Teste la logique de chevauchement horaire (surtout dans executer_tests).

- trouver_salles_alternatives() (main.py (line 209))
Propose des salles de remplacement quand la salle demandée est occupée.

- ajouter_dans_historique() (main.py (line 197))
Trace les actions importantes (création, annulation, etc.) pour audit/suivi.

- annuler_reservation() (main.py (line 230))
num = valeurs[0] peut être une chaîne selon Tkinter. Si les clés sont des int, l'annulation peut échouer silencieusement. Convertir en int + gérer le cas introuvable avec message.

- faire_reservation() (main.py (line 79))
Validation très fragile (boucles caractère par caractère) et message salle codé en dur (101-105 ou 201-205). À remplacer par validation robuste (int() + try/except) et message basé sur SALLES.

- executer_tests() (main.py (line 534))
Test "9 alternatives" est figé (main.py (line 628)). Si SALLES change, test faux négatif. Utiliser len(SALLES) - 1.

- afficher_historique() (main.py (line 344))
Conversion en majuscules faite manuellement en ASCII (main.py (line 376) à main.py (line 382)), ce qui gère mal les accents. Utiliser type_action.upper().

- sauvegarder_donnees() (fonctions.py (line 167))
except: est trop large et masque les erreurs disque/JSON. Il faut capturer l'erreur, la journaliser, et faire remonter un message clair à l'UI.

- charger_donnees() (fonctions.py (line 197))
Même problème de except: global, et en cas d'erreur ça repart silencieusement en données par défaut. Risque de "perte apparente" de données.